Arduino UNO R4 WiFi Real-Time Clock

Learn how to access the real-time clock (RTC) on the UNO R4 WiFi.

Author · Karl Söderby Last revision · 10/07/2025

In this tutorial you will learn how to access the real-time clock (RTC) on an **Arduino UNO R4 WiFi** board. The RTC is embedded in the UNO R4 WiFi's microcontroller (RA4M1).

Goals

The goals of this project are:

- Set a start date of the RTC
- Access the date / time from the RTC in calendar format.
- Access the time in Unix format.

Hardware & Software Needed

- Arduino IDE (online or offline)
- Arduino UNO R4 WiFi
- UNO R4 Board Package

Real-Time Clock (RTC)

The RTC on the UNO R4 WiFi can be accessed using the **RTC** library that is included in the **UNO R4 Board Package**. This library allows you to set/get the time as well as using alarms to trigger interrupts.

> The UNO R4 WiFi features a VRTC pin, that is used to keep the onboard RTC running, even when the boards power supply is is cut off. In order to use this, apply a voltage in the range of 1.6 - 3.6 V to the VRTC pin.

There are many practical examples using an RTC, and the examples provided in this page will help you get started with it.

Set Time

```
RTCTime startTime(30, Month::JUNE, 2023, 13, 37, 00, DayOfWeek::WEDNESDAY, SaveLight::SAVING_TIME_ACTIVE)
```

```
RTC.setTime(startTime)
```

To set the starting time for the RTC, you can create an `RTCTime` object. Here you can specify the day, month, year, hour, minute, second, and specify day of week as well as daylight saving mode.

Then to set the time, use the `setTime()` method.

```
#include "RTC.h"

void setup() {
  Serial.begin(9600);

  RTC.begin();

  RTCTime startTime(30

  RTC.setTime(startTim
}

void loop(){
}
```

Get Time

```
RTC.getTime(currentTime
)
```

To retrieve the time, we need to create a `RTCTime` object, and use the `getTime()` method to retrieve the current time.

This example sets & gets the time and stores it in an `RTCTime` object called `currentTime`.

```
#include "RTC.h"

void setup() {
  Serial.begin(9600);

  RTC.begin();

  RTCTime startTime(30

  RTC.setTime(startTim
}

void loop(){
RTCTime currentTime;

// Get current time fr
RTC.getTime(currentTim
}
```

Print Date & Time

The above examples show how to set & get the time and store it in an object. This data can be retrieved by a series of methods:

- `getDayOfMonth()`
- `getMonth()`
- `getYear()`
- `getHour()`
- `getMinutes()`
- `getSeconds()`

The example below prints out the date and time from the `currentTime` object.

```
#include "RTC.h"

void setup() {
  Serial.begin(9600);

  RTC.begin();

  RTCTime startTime(3

  RTC.setTime(startTi
}

void loop() {
  RTCTime currentTime

  // Get current time
  RTC.getTime(current

  // Print out date (
  Serial.print(curren
  Serial.print("/");
  Serial.print(Month2
  Serial.print("/");
  Serial.print(curren
  Serial.print(" - ")

  // Print time (HH/M
  Serial.print(curren
```

Unix

```
currentTime.getUnixTime
()
```

To retrieve the Unix timestamp, use the `getUnixTime()` method.

```
#include "RTC.h"

void setup() {
  Serial.begin(9600);

  RTC.begin();

  RTCTime startTime(30

  RTC.setTime(startTim
}

void loop() {
  RTCTime currentTime;

  // Get current time
  RTC.getTime(currentT

  //Unix timestamp
  Serial.print("Unix t
  Serial.println(curre

  delay(1000);
}
```

Periodic Interrupt

A periodic interrupt allows you to set a recurring callback.

To use this, you will need to initialize the periodic callback, using the `setPeriodicCallback()` method:

```
RTC.setPeriodicCallback
(periodic_cbk,
Period::ONCE_EVERY_2_SE
C)
```

You will also need to create a function that will be called:

```
void periodicCallback()
{ code to be executed }
```

Note the IRQ has a very fast execution time. Placing a lot of code is not a good practice, so in the example below we are only switching a single flag, `irqFlag`.

The example below blinks a light every 2 seconds:

```
#include "RTC.h"

volatile bool irqFlag
volatile bool ledStat

const int led = LED_B

void setup() {
  pinMode(led, OUTPUT

  Serial.begin(9600);

  // Initialize the R
  RTC.begin();

  // RTC.setTime() mu
  // what date and ti
  RTCTime mytime(30,
  RTC.setTime(mytime)

  if (!RTC.setPeriodi
    Serial.println("E
  }
}

void loop(){
  if(irqFlag){
    Serial.println("T
```

The period can be specified using the following enumerations:

- `ONCE_EVERY_2_SEC`
- `ONCE_EVERY_1_SEC`
- `N2_TIMES_EVERY_SEC`
- `N4_TIMES_EVERY_SEC`

`N32_TIMES_EVERY_SEC`

`N64_TIMES_EVERY_SEC`

`N128_TIMES_EVERY_SEC`

`N256_TIMES_EVERY_SEC`

Alarm Callback

```
RTC.setAlarmCallback(al
arm_cbk, alarmtime, am)
```

```
unsigned long previou
const long interval =
bool ledState = false

// Include the RTC li
#include "RTC.h"

void setup() {
  //initialize Serial
  Serial.begin(9600);

  //define LED as out
  pinMode(LED_BUILTIN

  // Initialize the R
  RTC.begin();

  // RTC.setTime() mu
  // what date and ti
  RTCTime initialTime
  RTC.setTime(initial

  // Trigger the alar
  RTCTime alarmTime;
  alarmTime.setSecond

  // Make sure to onl
  AlarmMatch matchTim
```

Network Time Protocol (NTP)

To retrieve and store the current time, we can make a request to an

will retrieve the UNIX time stamp and store it in an `RTC` object.

Code Source on Github

```
/**
 * RTC_NTPSync
 *
 * This example show
 * to the current da
 * Then the current
 *
 * Instructions:
 * 1. Download the N
 * 2. Change the WiF
 * 3. Upload this sk
 * 4. Open the Seria
 *
 * Initial author: S
 *
 * Find the full UNO
 * https://docs.ardu
 */

// Include the RTC l
#include "RTC.h"

//Include the NTP li
#include <NTPClient.

#if defined(ARDUINO_
#include <WiFiC3.h>
#elif defined(ARDUIN
```

Please also note that you will need to create a new tab called `arduino_secrets.h`. This is used to store your credentials. In this file, you will need to add:

```
#define SECRET_SSID ""
#define SECRET_PASS ""
```

Summary

This tutorial shows how to use the RTC on the UNO R4 WiFi, such as setting a start time, setting an alarm, or obtaining time in calendar or unix format.

Read more about this board in the **Arduino UNO R4 WiFi documentation**.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?